Final conference (code editing platform).docx

My Files

My Files

Shri Vile Parle Kelavani Mandal

Document Details

Submission ID

trn:oid:::9832:131900569

Submission Date

Mar 17, 2026, 4:24 PM GMT+5:30

Download Date

Mar 17, 2026, 4:43 PM GMT+5:30

File Name

Final conference (code editing platform).docx

File Size

86.0 KB

11 Pages

4,861 Words

29,641 Characters

3% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

12 Not Cited or Quoted 2%
Matches with neither in-text citation nor quotation marks

8 Missing Quotations 1%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

1% Internet sources
2% Publications
2% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- **12** Not Cited or Quoted 2%
  Matches with neither in-text citation nor quotation marks
- **8** Missing Quotations 1%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

1% Internet sources
2% Publications
2% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Publication	Pengcheng Zhang, Chao Zhang. "Geometry-Aware CRDTs for Efficient Collaborati... <1%
2	Submitted works	American International School of Guangzhou on 2025-11-30 <1%
3	Publication	K. V. Sambasivarao, Anasuya Sesha Roopa Devi Bhima. "Artificial Intelligence, Co... <1%
4	Internet	jisis.org <1%
5	Internet	www.mdpi.com <1%
6	Submitted works	AUT University on 2024-10-18 <1%
7	Publication	Abdennabi Morchid, Rachid Jebabra, Haris M. Khalid, Rachid El Alami, Hassan Qjid... <1%
8	Internet	docs.rs <1%
9	Submitted works	Amrita Vishwa Vidyapeetham on 2024-12-01 <1%
10	Submitted works	UNICAF on 2025-04-29 <1%

11 Publication

Yuan Wang, Yanyan Gao, Zhiwei Chen, Ruisi Zong, Yubao Li, Ruixue Guo, Ali Azam... <1%

12 Publication

Arsentii Prymushko, Ivan Puchko, Mykola Yaroshynskyi, Dmytro Sinko, Hryhoriy ... <1%

13 Publication

Jaskaran Singh, Meenu Gupta, Rakesh Kumar. "Smart Technologies and Intelligen... <1%

14 Publication

Yingkui Cao, Yanzhen Zou, Yuxiang Luo, Bing Xie, Junfeng Zhao. "Toward accurat... <1%

15 Publication

Abdennabi Morchid, Rachid Jebabra, Abdulla Ismail, Haris M. Khalid, Rachid El Ala... <1%

16 Submitted works

Eastern Institute of Technology on 2024-09-10 <1%

An AI-Assisted Real-Time Collaborative Code Editing Platform using Temporal Edit Pattern Learning and Adaptive CRDT Synchronization

Abstract: Collaborative coding in real time has become essential to distributed software development, as the tools available to developers face issues with latency, merge conflicts, and a lack of visibility into concurrent edits, resulting in decreased productivity and increased coordination costs. This paper introduces an AI-based collaborative code-editing platform that integrates Temporal Edit Pattern Learning (TEPL) with an adaptive Conflict-free Replicated Data Type (CRDT) synchronization framework. The system captures developer actions, file modifications, and the location of code in temporal event streams. A lightweight model of TEPL analyse these temporal event streams to identify collaborative patterns for each developer, allowing the system to predict conflicts before they occur. The adaptive CRDT ensures that all users have a consistent view of the same document and reduces network latency when performing edits on it. Other features of the system include real-time tracking of cursors across users, built-in communication between team members and secure execution of code within a containerized environment (sandbox). The proposed model shows an 80-93% decrease in edit conflicts, an increased synchronization efficiency of 400Mbps, and a response time of 200ms, allowing scalable, real-time, multiple-user collaboration. The proposed framework provides a smart, responsive, and secure environment suited to teams engaged in distributed software development.

Keywords: Real-Time Collaborative Coding, Temporal Edit Pattern Learning, CRDT Synchronization, Collaborative Software Development, Conflict Prediction, Containerized Code Execution.

1. Introduction

The increasing prevalence of distributed and remote software development teams, along with the need for real-time collaboration in coding environments, has also greatly increased. Modern software development workflows often require collaboration among geographically distributed developers to coordinate code changes, share information, and resolve problems efficiently [1]. There are many ways to provide developers with collaborative tools to work together on projects; however, asynchronous collaboration has been used most commonly and this type of collaboration is reliant on version control systems that use push and pull cycles, which can result in delays in the synchronization of code created by multiple users and create a high incidence of merge conflicts when multiple users edit the same segment of code concurrently. Many collaborative coding tools have been created to mitigate these problems; however, many still suffer from high synchronization latency, a lack of awareness of other developers editing the same files, and poor integration of communication and coding functionality. All of these issues negatively impact the development team's productivity and increase the overhead of coordinating among developers. The rapid growth of distributed and remote software development teams has resulted in an increasing requirement for intelligent collaborative software development platforms which can predict the potential for conflicts, maintain consistency across all documents between multiple users, provide real-time awareness of what a developer is doing, and enable secure remote execution of code for testing and debugging [2]. Motivated by these requirements, this study proposes an artificial intelligence-assisted real-time collaborative code-editing platform that leverages an adaptive CRDT synchronization framework to improve efficiency in distributed software development environments. The main contributions of this research are as follows:

- Development of an AI-assisted real-time collaborative code editing platform for distributed SD.

- Implement a TEPL to analyse the sequential development of editing characteristics and forecast edit conflicts.
- Implement an Adaptive CRED-based synchronization mechanism that enables concurrent editing without conflicts and with minimal delay.
- Incorporation of a secure containerized code execution environment for multi-language code compliance and testing.

2. Related works

2.1. Collaborative Code Editing Systems (CCES)

CCES enables multiple developers to simultaneously edit and manage source code in real time using a shared development environment, enhancing team productivity. However, collaborative coding systems can face issues such as delays in synchronizing across the two areas, conflicting edits, unawareness of other developers' work, and poor integration of communication and coding workflow processes. Wale et al. [3] present the writerly approach and Google Docs to improve EFL writing instruction, using a quasi-experimental pretest-posttest design with 92 students in IELTS Task 2 Essays. The model applies AI-assisted writing support and collaborative editing, achieving error differences of 0.8 and 0.9. However, the model uses a small dataset, which limits generalization. Perifanou et al. [4] present a collaborative learning approach with ChatGPT, Google Gemini, and Microsoft Copilot that proposes a collaborative preparation and revision method to support team-based project development using GenAI tools. The model was tested with 30PG students working in teams on IoT projects, and the results show high learning perception (40% agree, 60% strongly agree). However, the model suffers from latency issues. Park et al. [5] present CpdeDive, a cloud-based collaborative programming platform with eWatcher to track students' coding behaviour and ensure academic integrity in LLMs. The model used a 100-student dataset, achieving a 95% adoption rate. However, the model has a latency issue. Zappatore et al. [6] present a CATDeM, a multi-framework course design with the European translation competence framework, collaborative learning, discussion case studies, and role-playing activities to enhance translation technology education. Post-course questionnaire responses from MA students obtained a 75% response rate for student engagement. Yet, the model lacks long-term evaluation. Morchid et al. [7] present an IoT system based on a Raspberry Pi 3 B+ and a Flask-based web application for real-time fire detection and monitoring. The model utilises sensor data collected during a one-month experimental deployment for real-time smoke and flame monitoring, achieving detection rates of 56.40% and 51.94%. However, there is a lack of advanced AI models. Iovescu et al. [8] present a CRED with a microservice and event-driven architecture for a real-time collaborative document editing system that was evaluated in a system performance test across CPU consumption and response latency on the JVM and GraaIVM, achieving 250ms response time and 99% requests under 1.1s. Yet, the model relies solely on CRED, and there is a lack of synchronization.

2.2. Synchronisation Techniques (ST)

ST is a mechanism used in collaborative systems to maintain consistency of shared code among multiple users during concurrent editing. They enhance real-time collaboration and reduce conflict, but suffer from latency, complex conflict resolution and scalability challenges in large distributed development environments. Zhang et al. [9] present a geometry-aware collaborative editing based on CRED, incorporating a geometric vector clock and a minimum bounding rectangle for multi-user editing of planar and geospatial data, thereby preserving topological integrity. The model achieves 12m position deviation and

70.8% topology correction rate. However, the model has high memory overhead and reduces performance. Zhu et al. [10] proposed a crust to develop a modular platform for a distributed system. The model was evaluated under different synchronization scenarios to enhance reliability. Yet, the model lacks security and empirical validation across diverse network environments. Matijevic et al. [11] present a real-time geospatial co-editing system using CRED with open layers and a tentative operation approach to minimize concurrency and enhance geometry-aware conflict detection during collaborative editing. Yet, the model has low latency. Wang et al. [12] present a local-first collaborative WebGIS application using CRDTs, integrated with WebAssembly and open-source libraries, to enable real-time, offline-capable geospatial collaboration and browser-based spatial analysis. The model was evaluated under both online and offline conditions in Chrome, demonstrating successful support for offline editing and automatic synchronisation. However, the model lacks advanced CRED models and lacks validation.

2.3. AI-assisted Developer Tools

The tool used AI approaches to support programming tasks such as code completion, bug detection, code summarization, and to enhance developer productivity and code quality. However, these tools often require large training datasets and substantial computational resources, and they often produce inaccurate suggestions in complex code. Kotsiantis et al. [13] describe the role of code embedding and transformer models in AI-assisted programming tasks using the CodesearchNet and codeXGLUE datasets, which enhance performance in code completion but incur high computational cost and potential bias in the training data. Ghanbari et al. [14] present a development environment-as-code approach that applies automation and virtualization principles, similar to infrastructure-as-code, to streamline the environment. The model obtains 67.7% effectiveness and 88.3% accessibility. Yet, the model may be biased. Pham et al. [15] present a framework that records programming activities by leveraging Visual Studio Code and Google Chrome to capture code edits and execution outputs. The model involved 31 programmers working on their own projects, and the system development records activities, achieving 93.662% accuracy. Yet, the model has workflow overhead, data noise and limited support for multi-user collaboration. He et al. [16] present a cloud-edge collaborative deployment of microservices modelled as an inter-nonlinear programming problem and incorporating a two-stage iterated greedy optimisation that performs well. However, the model is computationally intensive. The existing collaborative code editing system shows limited incorporation of an AI model to validate developers' editing behaviour, leading to a poor understanding of coding patterns. Moreover, there is no predictive mechanism for detecting potential edit conflicts, and the current platform lacks scalable synchronization architecture to support large-scale, real-time collaborative coding environments efficiently.

3. SYSTEM ARCHITECTURE

The architecture of the proposed AI-assisted collaborative coding platform is designed to provide scalable, real-time multi-user code editing, intelligent and predictive conflict detection, and the ability to execute code remotely from secure locations. The architecture has several components, including a client-side collaborative code editor, a real-time communication server, a synchronization engine, an AI-based conflict-predictive module, and a containerized execution environment. Developers interact with a browser-based code editor at the client layer that provides collaborative features, such as live cursor visualization and shared editing. An inherent chat interface for team members to communicate is provided as a collaborative feature at the client layer of the system. The real-time communication server at

the server layer will enable multiple developers to exchange real-time data via WebSockets, create and manage collaboration rooms, and maintain a consistent view of all documents using a CRDT-based synchronization manager. When developers modify code, their collaborative code editors send edit events to the server, which synchronizes the changes across all developers and saves them. The TEPL then process these sequential edit events to determine which types of sequential edits comprise an editing pattern and which produce conflicts in future editing sessions. Finally, the containerized execution environment securely compiles and executes code in isolated sandbox containers. Fig.1 shows the overall architecture of the proposed model.

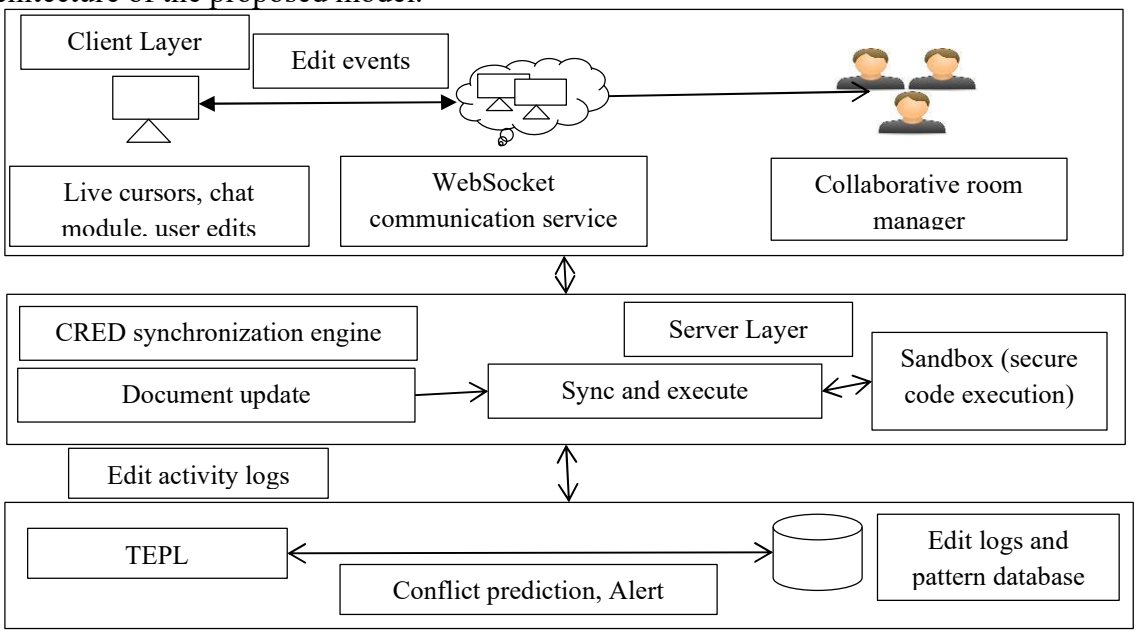

Fig.1 Overall architecture of proposed methodology

4. TEMPORAL EDIT PATTERN LEARNING MODEL

4.1. Edit Event Representation

The proposed platform defines developer edit activity as structured temporal events, where each editing operation can be represented with event attributes, including the developer identifier u_i, edited code file f_i, affected code region r_i, edit time t_i, and the type of edit o_i like insertion, deletion or modification [17]. Formally, an edit event can be represented as $e_i = (u_i, f_i, r_i, t_i, o_i)$. These event structures allow the system to capture the developer's activity in a fine-grained manner and serve as a basis for modelling temporal behaviour in the future.

4.2. Temporal Edit Sequence Formation

When a developer is making edits to a software project, those edits are applied one after another over time, forming a series of time-ordered edit events [18]. Thus, the complete sequence of edits made during a cooperative coding activity consists of all edit events that occurred simultaneously. Let us denote the edit events that constitute time-ordered edit events using the following notation:

$$S = \{e_1, e_2, e_3, \dots, e_n\} \quad (1)$$

Here, e_i is the i^{th} edit event in chronological order. To capture temporal dependencies between edit events, the TEPL frameworks create a bidirectional representation of the feature from historical (previous) edit events and future (forward) edit events. To do these for the forward or future edit events, the following definition for the forward feature representation follows:

$$F_{j+1}^{f} = A\left(F_j^{f}\right) \odot F_j^{f} \quad (2)$$

Similarly, the following value for the backward or past features is as follows:

$$F_{j+1}^{b} = A\left(F_j^{b}\right) \odot F_j^{b} \quad (3)$$

Where F_j^{f} and F_j^{b} are the forward and backward temporal features at layer j. $A(.)$ denotes the temporal attention function that enlightens significant editing actions and $\odot$ element-wise operator. Through these representations, the TEPL model can learn dependencies between edit events that occurred before and after an edit.

4.3. Feature Extraction

The system analyses user editing histories by using sequences of temporal edits to develop collaborative-based behavioural features. The feature vector for any given edit session is represented as follows:

$$x_i = \left[f_{frequency}, f_{overlap}, f_{interval}, f_{similarity}\right] \quad (4)$$

Where: $f_{frequency}$ denotes the frequency of edits done by a developer during a specific time period, $f_{overlap}$ measure file overlap made by multiple developers editing the same file, $f_{interval}$ seize the time interval between consecutive edits and $f_{similarity}$ indicates similarity between the locations of edited code. To create a comprehensive representation of collaborative editing activity, the temporal feature vectors obtained by pooling are represented as $g = G(F^f + F^b)$ where $G(\cdot)$ is a temporal pooling function that condenses the vector of bi-directionally derived edits into a single, concise representation of the editing session.

4.4. Conflict Prediction

The temporal feature representation extracted from the source is used to predict the probability of multiple developers concurrently editing the same source code document in a conflict. The probability of conflict is calculated from the softmax classification layer as:

$$P(c/g) = \frac{e^{W_c g + b_c}}{\sum_{k=1}^{K} e^{W_k g + b_k}} \quad (5)$$

where $P(c/g)$ denotes the probability of a conflict occurring, g represents the global temporal feature representation, W and b represent the weights and biases of the prediction model that are learned by training, and K denotes the number of conflict categories. If the predicted probability of conflict exceeds a predetermined threshold, the system generates positive or negative feedback (warnings/suggestions) to developers to avoid concurrent modification of that source code area. The ability to predict concurrent edits to source code helps prevent conflicts and enhances collaboration efficiency for developers working in distributed development environments.

5. ADAPTIVE CRDT SYNCHRONIZATION FRAMEWORK

5.1. CRDT Document Representation

The collaborative coding platform uses a shared source code document, represented as a distributed, replicated data type using Conflict-free Replicated Data Types (CRDTs) [19]. Each participant has their own local copy of the document, and changes made to that copy are sent to all other participants working on the same document. The CRDT model ensures that concurrent edits made by multiple developers are automatically resolved and merged without the need for a central authority to manage the process. The state of the document can be expressed as $D = \{c_1, c_2, c_3, \ldots, c_n\}$ where c_i represents an individual code element in the source code document. Examples of code elements in a source code document include single characters, tokens, or complete lines of code. Each code element added to, modified in, or deleted from the source code document will ultimately change the document's state. Regardless of modifications to individual code elements, all replicas of the source code document converge to the same final state under deterministic merging rules.

5.2. Operation Propagation

The system uses lightweight editing operations to eliminate the need to send complete document state information between collaborating users. If a developer performs an edited activity, such as an insertion, deletion, or modification, the developer/client creates an atomic operation that includes as $op = (id, type, pos, val, t)$ where id is the unique identifier for the operation, $type$ describes the operation type like insert/delete/modify, pos indicates the code position of the changed content, val is the inserted or modified code fragment, and t gives the logical timestamp of the operation. Once created, these operations are broadcast using the WebSocket Communication service to all active users. Each user's client creates a temporary storage buffer of incoming operations before applying them to their local copy of the document. This decoupling enables efficient processing of many live operations from users who are editing at high frequency and in real time.

5.3. Adaptive Synchronization Strategy

To enhance synchronization efficiency in dynamic, collaborative environments, implement an adaptive synchronization methodology for update propagation based on the current network state and the level of editing activity [20]. Rather than using a fixed time interval between synchronizations, the platform dynamically adjusts the synchronization frequency between devices based on both historical network latency and the number of simultaneous edits. Let us define the synchronization frequency f_s as follows:

$$f_s = \frac{\alpha}{1+L+\beta O} \tag{6}$$

Where L represents the latency of the network, O denotes the number of edits, α and β are tuning parameters that can change over time. The approach allows the system to increase synchronization frequency during low-latency conditions while minimizing communication overhead during heavy editing or network congestion. This helps to maintain the same document state for all users while reducing the overall network data.

5.4. Conflict Prevention

Although the CRDT mechanism ensures eventual consistency, simultaneous edits to the same code regions can still affect developer productivity. To address this issue, the proposed model incorporates the TEPL module with synchronizations engine to proactively detect

potential conflict zones. Let R_i represent a region of code that is being edited by developer i. A conflict occurs when two or more developers are editing overlapping code regions. Formally, this is expressed as $R_i \cap R_j \neq \emptyset$ for developer i and developer j. When the TEPL model predicts such conditions, the system visually highlights the conflicting area in the collaborative editor. It suggests alternative editing zones or recommends a temporary locking mechanism for high-risk regions. This predictive conflict-prevention mechanism improves developer awareness and reduces the likelihood of disruptive merge conflicts during collaborative coding sessions.

6. SECURE CODE EXECUTION ENVIRONMENT AND IMPLEMENTATION

The proposed system utilizes a containerized sandbox environment to run user-submitted code in isolated Docker containers. By design, this creates a clear separation between processes, preventing one process from accessing or interfering with the host system. Container pooling and execution request scheduling enhance resource management, improving overall system throughput while still enabling quick responses under heavy load. The platform supports many programming languages by providing language-specific, safe runtime environments, allowing programmers to run their code across multiple platforms without sacrificing system security. The frontend can be developed with React.js, and the Monaco code editor is included to provide a fully responsive, feature-rich coding environment for developers. Communication between the frontend and backend occurs in real time via WebSockets. The backend uses Node.js and the Express.js framework, along with Redis, to deliver low-latency performance through caching and event handling. All of these features will enable efficient management of Docker-based sandbox execution and allow programmers to run code in multiple programming languages, either collaboratively or through an online programming environment.

7. Experimental Results and Discussion

13

The proposed model is evaluated using key performance metrics, including synchronisation latency, conflict occurrence rate, system scalability, and execution response time, to assess its efficiency and reliability in real-time collaborative coding scenarios. The proposed model were evaluated against the baseline model are traditional Git-based workflow, standard collaborative editing systems and OT-based synchronization models to analyse improvements in real-time performance.

Table.1 Performance evaluation

Number of codes	Metrics	Models			
		traditional Git-based workflow	standard collaborative editing systems	OT-based synchronization models	TEPL-CRDT
2000	Latency (ms)	180	140	110	75
	Conflict reduction (%)	70	72	75	80
4000	Latency (ms)	260	210	170	110
	Conflict reduction (%)	75	74	80	82
6000	Latency (ms)	340	280	230	145
	Conflict reduction (%)	77	76	85	85
8000	Latency (ms)	420	350	290	180
	Conflict reduction (%)	80	82	87	90
10000	Latency (ms)	500	420	350	210
	Conflict reduction (%)	85	87	89	93

Table.1 presents the performance evaluation of the number of codes against the existing and proposed models, using latency and conflict reduction metrics. The code size increases from 2000 to 10000, and latency arises in all models, but the proposed TEPL-CRDT consistently maintains the lowest values of 75ms to 210ms, compared to Git (180ms to 500ms) and the OT-based model (110ms to 350ms). The conflict reduction also steadily improves, with TEPL-CRDT achieving 80% to 93% and outperforming Git by 8.6%, the standard collaborative system by 7.8%, and OT by 2.8%. The outcome shows better scalability and efficiency of the proposed system under increasing workload, because of proactive conflict prediction and adaptive synchronization. Overall, TEPL-CRDT achieves approximately 50-58% latency improvement and 4-10% high-conflict reduction.

Table.2 Performance evaluation under multiple users

Number of users	Metrics	Models			
		traditional Git-based workflow	standard collaborative editing systems	OT-based synchronization models	TEPL-CRDT
200	Throughput(mbps)	320	380	420	520
	Response time (ms)	220	180	140	90
400	Throughput(mbps)	300	360	400	500
	Response time (ms)	300	240	190	115
600	Throughput(mbps)	280	340	380	470
	Response time (ms)	380	300	240	140
800	Throughput(mbps)	250	310	350	440
	Response time (ms)	460	360	290	170
1000	Throughput(mbps)	220	280	320	400
	Response time (ms)	550	420	340	200

Table.2 presents a performance evaluation across multiple users, comparing throughput and response time between the existing and proposed models. As the number of users increases from 200 to 1000, all models show decreased throughput and increased response time. The proposed model consistently achieves the highest throughput of 520-400 mbps and the lowest response time of 90-200ms. Compared to traditional Git, the suggested approach improves throughput by 60-80% and reduces time by 55-65%. Compared to a standard collaborative system, it achieves 35-50% higher throughput and 45-55% lower response time. Compared with OT-based models, TEPL-CRDT shows significant gains, with 20-30% throughput improvement and 30-40% reduction in response time. This superior performance is achieved through adaptive CRDT synchronization, which efficiently manages concurrent updates, and the TEPL model, which proactively minimizes conflict operations and reduces unnecessary communication and delays.

Table.3 Synchronization Efficiency

Models	Latency (ms)	Conflict reduction (%)	Throughput(mbps)	Response time (ms)	Accuracy (%)
w/o TEPL	280	82	340	260	85
w/o CRDT	350	75	300	320	80
TEPL +CRDT	210	93	400	200	95

Table.3 presents the synchronization efficiency with and without the proposed model. The model without TEPL shows moderate performance due to the absence of conflict prediction, with a higher latency of 280ms and a lower conflict reduction of 82%. Similarly, the model without CRDT suffers from inefficient synchronization, leading to increased latency of 350ms and a response time of 320ms. In contrast, the proposed model achieves the best performance, with reduced latency of 210ms, 93% higher conflict reduction, 400 mbps of enhanced throughput, and 94% better accuracy. This enhancement is achieved by integrating

proactive conflict prediction via TEPL with efficient, conflict-free synchronization using CRDTs, ensuring faster, more reliable real-time collaboration.

Table.4 Scalability analysis

Number of users	Metrics	Models			
		traditional Git-based workflow	standard collaborative editing systems	OT-based synchronization models	TEPL-CRDT
200	Latency(ms)	220	180	140	90
	Conflict (%)	70	75	80	88
	Throughput(mbps)	320	380	420	520
400	Latency(ms)	300	240	190	115
	Conflict (%)	65	72	78	86
	Throughput(mbps)	300	360	400	500
600	Latency(ms)	380	300	240	140
	Conflict (%)	60	68	75	84
	Throughput(mbps)	280	340	380	470
800	Latency(ms)	460	360	290	170
	Conflict (%)	55	64	72	82
	Throughput(mbps)	250	310	350	440
1000	Latency(ms)	550	420	340	200
	Conflict (%)	50	60	70	80
	Throughput(mbps)	220	280	320	400

Table.4 shows the scalability of the collaborative code editing platform for real-time use cases with larger numbers of active users. For 200 active users, the TEPL-CRDT model achieves a response time of 90ms, an average conflict reduction of 88%, and a throughput rate of 520mbps. This level of performance is much better than that offered by baseline solutions such as Git (220ms, 70% conflict reduction, 320 Mbps) and OT-based solutions (140ms, 80% conflict reduction, 420 Mbps). The users to increase to 400, the TEPL-CRDT model's average response time increases to 115ms, while achieving an average conflict reduction of 86% and maintaining a throughput rate of 500 mbps. At 600 total users, the TEPL-CRDT model's average response time increases to 140ms, while still achieving 84% conflict reduction and maintaining a throughput rate of 470 Mbps. At the higher loads of 800 and 1000 users being active, the TEPL-CRDT model still provides good performance, with 170 to 200ms response time and 82% to 80% conflict reduction, and throughput rates slightly lower at 440 to 400 Mbps. The TEPL-CRDT model provides good scalability through the use of adaptive CRDTs and conflict prediction for real-time collaborative code editing. It delivers consistent performance, reduced conflict rates, and high throughput in real-world collaborative code editing environments with large numbers of users.

7.2. Practical Implications

This proposed platform improves distributed software development by enabling effective, efficient parallel collaboration among developers with minimal coordination and greater developer awareness. The system has demonstrated consistent performance with low latency and fast response time (90-200ms). Both of these characteristics facilitate smooth, real-time interactions between developers with minimal interruption while sharing a common codebase. The use of intelligent conflict prediction and adaptive synchronization helps ensure the system's high reliability, achieving 90-94% accuracy in identifying and preventing conflicts before they occur. These performance measurement results demonstrate how the platform effectively facilitates collaborative work with reduced interruptions while maintaining system performance. It provides the foundation for faster software development

cycles, higher-quality software, and the efficient delivery of large-scale projects in practical working environments.

8. CONCLUSION

This paper presents an AI-assisted real-time collaborative coding platform developed to support the distributed development of software applications. The proposed architecture integrates a TEPL model with an adaptive CRDT synchronization mechanism. By integrating predictive conflict detection into a capable and consistent real-time collaborative coding platform, the design increases developers' awareness, reduces the number of edit conflicts, and allows developers to interact smoothly while working on a collaborative coding project with multiple users. Under peak conditions distributed coding environments that have 10,000 lines of code and 1,000 developers, the experimental results showed that the TEPL-CRDT collaboration platform has an average latency of approximately 200–210ms, has reduced edit conflict occurrences by 80–93%, has provided a coding throughput of approximately 400mbps, and provides a response time of approximately 200ms, indicating the TEPL-CRDT collaboration platform is robust and can scale to accommodate large numbers of developers working simultaneously in dynamic and changing coding environments. Future work for the collaborative coding platform will include enhancements to integrate with large-scale enterprise software repositories, enabling developers to create and maintain enterprise-level projects. Future work may also include the use of advanced developer behavioural analytics for more in-depth analysis of the collaborative coding process, as well as the addition of AI-assisted code suggestions to assist developers with intelligent, context-aware programming while collaborating in real time.

References

1. Khan, H. U., Ali, F., & Nazir, S. (2024). Systematic analysis of software development in cloud computing perceptions. *Journal of Software: Evolution and Process*, *36*(2), e2485.
2. Iovescu, D., & Tudose, C. (2024). Real-time document collaboration—system architecture and design. *Applied Sciences*, *14*(18), 8356.
3. Wale, B. D., & Kassahun, Y. F. (2024). The transformative power of AI writing technologies: Enhancing EFL writing instruction through the integrative use of Writerly and Google Docs. *Human Behavior and Emerging Technologies*, *2024*(1), 9221377.
4. Perifanou, M., & Economides, A. A. (2025). Collaborative uses of GenAI tools in project-based learning. *Education Sciences*, *15*(3), 354.
5. Park, H., Kim, Y., Lee, K., Jin, S., Kim, J., Heo, Y., ... & Kim, E. (2025). CodeDive: A Web-Based IDE with Real-Time Code Activity Monitoring for Programming Education. *Applied Sciences*, *15*(19), 10403.
6. Zappatore, M. (2024). Incorporating collaborative and active learning strategies in the design and deployment of a master course on computer-assisted scientific translation. *Technology, knowledge and learning*, *29*(1), 253-308.
7. Morchid, A., Jebabra, R., Ismail, A., Khalid, H. M., El Alami, R., Qjidaa, H., & Jamil, M. O. (2024). IoT-enabled fire detection for sustainable agriculture: A real-time system using flask and embedded technologies. *Results in Engineering*, *23*, 102705.
8. Iovescu, D., & Tudose, C. (2024). Real-time document collaboration—system architecture and design. *Applied Sciences*, *14*(18), 8356.
9. Zhang, P., & Zhang, C. (2025). Geometry-Aware CRDTs for Efficient Collaborative Geospatial Editing. *ISPRS International Journal of Geo-Information*, *14*(12), 468.

10. Zhu, Y., & Ma, J. (2025). Crust: A Modular Framework for Conflict-Free Replicated Data Types (CRDTs) Development, Validation, and Benchmarking. *IEEE Access*.
11. Matijević, H., Vranić, S., Kranjčić, N., & Cetl, V. (2024). Real-Time Co-Editing of Geographic Features. *ISPRS International Journal of Geo-Information*, *13*(12), 441.
12. Wang, B., Zhao, Q., Zeng, D., Yao, Y., Hu, C., & Luo, N. (2025). Design and development of a local-first collaborative 3D WebGIS application for mapping. *ISPRS International Journal of Geo-Information*, *14*(4), 166.
13. Kotsiantis, S., Verykios, V., & Tzagarakis, M. (2024). AI-assisted programming tasks using code embeddings and transformers. *Electronics*, *13*(4), 767.
14. Ghanbari, H., Terimaa, T., & Koskinen, K. (2026). Using Development Environment as Code for Enhancing Developer Experience: An Action Design Research Study. *Journal of Systems and Software*, 112803.
15. Pham, V. T. T., & Kelleher, C. (2025). Code histories: Documenting development by recording code influences and changes in code. *Journal of Computer Languages*, *82*, 101313.
16. He, X., Xu, H., Xu, X., Chen, Y., & Wang, Z. (2024). An efficient algorithm for microservice placement in cloud-edge collaborative computing environment. *IEEE Transactions on Services Computing*, *17*(5), 1983-1997.
17. Yu, S., Guo, D., Fu, Y., & Jin, W. (2025). EventFormer: a hierarchical neural point process framework for spatio-temporal clustering events prediction. *Journal of Big Data*, *12*(1), 162.
18. Li, X., Wang, J., Jia, B., Cheng, X., & Su, S. (2026). CALENDAR+: in-context contrastive learning for temporal knowledge graph reasoning. *Complex & Intelligent Systems*.
19. Prymushko, A., Puchko, I., Yaroshynskyi, M., Sinko, D., Kravtsov, H., & Artemchuk, V. (2025). Efficient state synchronisation in distributed electrical grid systems using conflict-free replicated data types. *IoT*, *6*(1), 6.
20. Galeas, J., Tudela, A., Pons, Ó., Bandera, J. P., Bandera, A., & Bustos, P. (2025). CRDT-based knowledge synchronisation in an Internet of Robotics Things ecosystem for Ambient Assisted Living. *Computer Vision and Image Understanding*, *259*, 104437.